

שים לב

שלוש דרגות בכל קושי. קרא את הדחיפה, נסה עשר דקות, ורק אז את האסטרטגיה. הכמעט-פתרון הוא הרשת, לא הקיצור. אחד עשר קשיים, מסודרים לפי הסדר שבו הם קורים בפועל — לא לפי סדר החלקים בברייף. הכלי שפותר את רובם הוא DevTools, לשונית Application, ו-Local Storage. שם רואים את המפתח, את הערך המדויק, ואפשר גם לערוך אותו ביד. פתח אותה עכשיו והשאר אותה פתוחה.

א · "שמרתי, רעננתי, וכלום לא חזר"

דחיפה. יש שתי אפשרויות ואפשר להפריד ביניהן בעשר שניות: או שאף אחד לא כתב, או שאף אחד לא קרא. תסתכל ב-Application אם המפתח בכלל קיים.

אסטרטגיה. אם המפתח לא שם — הבעיה בכתיבה: או ש-`persist` לא רשומה ב-`app.js`, או ש-`save` יוצאת מוקדם. אם המפתח כן שם ובתוכו הנתונים — הבעיה בקריאה: או ש-`load` מחזירה `null`, או ש-`app.js` לא קורא לה בכלל בשורה הראשונה.

כמעט-פתרון. אם בערך של המפתח כתוב בדיוק `[object Object]`, מצאת: `setItem` ממירה את הארגומנט השני למחרוזת, ומחרוזת שאובייקט מומר אליה היא חמש עשרה התווים האלה, תמיד, לכל אובייקט בעולם. הדבר שנשמר לא היה JSON.

ב · "העמוד לבן ובקונסולה שגיאה על `map` או על `length`"

דחיפה. מה מחזירה `load` בביקור הראשון, כשהמפתח עדיין לא קיים? תרשום את התשובה לפני שאתה בודק.

אסטרטגיה. `getItem` על מפתח שלא קיים מחזירה `null`. השורה שאחריה, אם היא `JSON.parse(raw)`, לא זורקת — היא מקבלת את המחרוזת "null" ומחזירה `null`. אז `load` מחזירה `null` פנימה למקום שמצפה למערך, והקריסה מגיעה ארבע שורות אחר כך, בתוך `render`, ונראית כמו באג ברינדור.

כמעט-פתרון. בדיקה אחת, לפני כל `parse`: אם מה ש-`getItem` החזירה הוא `null` — צא והחזר `null`. זה לא טיפול בשגיאה, זו התשובה הרגילה לביקור ראשון.

ג · "שמתי `try/catch` וזה עדיין נופל"

דחיפה. על מה בדיוק זה נופל? שים את המחרוזת שהפילה אותך בקונסולה ותריץ עליה `JSON.parse` ידנית. אם היא לא זרקה — ה-`try` שלך לא היה קשור לבעיה.

אסטרטגיה. `JSON.parse` שהצליח אומר שהמחרוזת הייתה תקינה. הוא לא אומר כלום על המבנה. שלושה מטענים תקינים לחלוטין שהיישום שלך צריך לשרוד: `null`, מערך של מספרים, ואובייקט שבו השדה שאתה מצפה שיהיה מערך הוא מחרוזת.

כמעט-פתרון. אחרי ה-`parse`, שאל שלוש שאלות בסדר הזה: זה אובייקט? יש בו מערך במקום שאתה מצפה? כל איבר בו נראה כמו פריט — כלומר יש לו מזהה שהוא מחרוזת לא ריקה, ומספר שהוא באמת מספר? בדוק את השדות שאתה באמת קורא, ולא יותר.

ד · "מה ההבדל בין null לבין מערך ריק, ולמה זה חשוב"

דחיפה. נסה: מחק את כל הפריטים, ואז רענן. מה אתה רואה?

אסטרטגיה. אם חזרו נתוני הדוגמה — יש לך את הבאג. null פירושו "לא נשמר כלום מעולם"; [] פירושו "המשתמש מחק הכול", וזו תשובה ולא היעדר תשובה. אם load מחזירה null בשני המקרים, מי שמחק את הכול מקבל את הדוגמאות בחזרה בכל רענון. **כמעט-פתרון.** חפש בקוד שלך תנאי כמו "אם אין אורך למערך, החזר null" או "אם המערך ריק, החזר null". אחד כזה מספיק. שים לב שזה לא קשור ל- ?? מול || — מערך ריק הוא truthy, ולכן שני האופרטורים מתנהגים כאן אותו דבר.

ה · "מה בדיוק שומרים"

דחיפה. תסתכל על אובייקט ה-state שלך ושאל על כל שדה: אם אאבד את זה כשסוגרים את הלשונית — מישהו ייפגע?

אסטרטגיה. שומרים את מה שאי אפשר לחשב מחדש ו**שחשוב שיישאר**: האוסף. לא שומרים את מה שנגזר (זו כבר עבודה של שבוע 9), ולא שומרים את מה ששייך ל**ישיבה הזאת** — מה שהוקלד בחיפוש, המיון הנוכחי, איזו שורה פתוחה בטופס. **כמעט-פתרון.** נסה לשמור את החיפוש ותראה למה לא: הקלד משהו בחיפוש שמצמצם לשורה אחת, סגור את הלשונית, ופתח מחר. תראה פריט אחד ולא תזכור למה. זה הבאג שנקרא בדיוק כמו "הנתונים שלי נעלמו", והוא גם הכי קשה לדיווח.

ו · "איזה שם לתת למפתח"

דחיפה. פתח את Application, Local Storage, והסתכל מה כבר יושב שם מהפרויקטים שלך.

אסטרטגיה. localStorage משויך ל-origin — סכימה, מארח ופורט — ולא לתיקייה. כל דבר שאתה מריץ עם Live Server מוגש מאותו http://127.0.0.1:5500. מפתח בשם items הוא מפתח משותף לכל מה שכתבת השנה. **כמעט-פתרון.** שלושה חלקים: של מי, של מה, ואיזו צורה יש בפנים. משהו בסגנון ruppin.pantry.v1. החלק השלישי הוא זה שנראה מיותר היום ומציל אותך בחלק ד.

ז · "הטופס לא מתמלא כשאני לוחץ על עריכה"

דחיפה. מי אמור למלא אותו — render או המטפל?

אסטרטגיה. המטפל. render רץ בכל הקשה בתיבת החיפוש, ושדה שהערך שלו נדרס בזמן שמישהו מקליד בו הוא שדה שנלחם במשתמש. מה ש-render כן מצייר הוא **המצב** של הטופס: תווית הכפתור, כפתור הביטול, והסימון על השורה שנערכת. **כמעט-פתרון.** הכלל, ושווה לזכור אותו לפרויקט: **נתונים מצוירים מ-state; הערך של פקד — לא.** אותה הבחנה בדיוק שהייתה כבר בשבוע 9 סביב ה-select של המדפים.

ח · "העריכה מוסיפה שורה חדשה במקום לשנות"

דחיפה. בשליחה, איך אתה יודע אם זו הוספה או עריכה?

אסטרטגיה. לפי השדה החדש ב-state — אם הוא null, זו הוספה; אם יש בו מזהה, זו החלפה. שתי הדרכים מסתיימות ב-setState אחת, והן נבדלות בשורה אחת.

כמעט-פתרון. ההחלפה היא .map. על האוסף שמחזירה אובייקט חדש לפריט הנערך ומשאירה את כל השאר כמו שהם. את המזהה ואת תאריך היצירה לוקחים מהפריט הישן — הם לא בין השדות שהטופס מחזיר, ולכן אין דרך לעורך לשנות אותם בטעות.

ט · ניקיתי הכול, רעננתי, והדוגמאות חזרו

דחיפה. מה בדיוק עשית ב"נקה הכול"? אם התשובה היא `removeItem` — מצאת.

אסטרטגיה. מפתח שאינו קיים אומר ליישום "הדפדפן הזה מעולם לא היה כאן", והיישום זורע בשמחה את נתוני הדוגמה. **לנקות את הנתונים של המשתמש פירושו לשמור אוסף ריק**, לא למחוק את המפתח.

כמעט-פתרון. המטפל של "נקה הכול" לא צריך לגעת ב- `localStorage` בכלל. הוא מבקש אישור ואז עושה `setState` עם אוסף ריק — והמנוי כותב את זה לבד. `removeItem` שייך למקום אחד ואחד בלבד: לכפתור שמשמעותו "תעשה כאילו לא הייתי כאן", כלומר ההצעה לשחזר את נתוני הדוגמה.

י · "המיגרציה — לא ברור לי איך בכלל בודקים אותה"

דחיפה. אתה לא צריך גרסה קודמת אמיתית. פתח את Application וכתוב אחת ביד.

אסטרטגיה. קח את הערך שיש לך במפתח, העתק ממנו רק את המערך שבפנים, והדבק אותו במקום הערך המלא. זה בדיוק המבנה שהניסיון הראשון שלך הבוקר כתב. רענן: היישום חייב לאמץ אותו, לא לזרוק אותו.

כמעט-פתרון. שלוש תשובות ולא שתיים. המבנה הנוכחי — קח את האוסף. מערך חשוף — אמצ. כל השאר — `null`, וזרע. השלישית היא זו שדורשת אומץ: גרסה שלא מוכרת עלולה להיות שונה בכל שדה, וניחוש **משחית** את הנתונים במקום לאבד אותם, וזה גרוע יותר.

יא · "התיקון בחלק ה — לא הבנתי מה מודדים"

דחיפה. כמה פעמים נכתב לדיסק כשהקלדת מילה בת שש אותיות בתיבת החיפוש? נחש, ואז תמדוד.

אסטרטגיה. עוטפים את `setItem` במונה. אפשר ישירות בקונסולה: שומרים את הפונקציה המקורית בצד, שמים במקומה אחת שמגדילה מונה וקוראת למקורית, ואז מקלידים ומסתכלים. אין צורך לגעת בקוד של היישום כדי למדוד אותו.

כמעט-פתרון. שתי דרכים לתקן, ושתיהן נכונות. הזולה: שמור **בהפניה** את המערך האחרון שנכתב, וצא מוקדם אם הוא אותו מערך בדיוק. זה עובד רק בזכות כלל משבוע 9 שאין לו שום קשר לאחסון — המטפלים בונים מערכים חדשים במקום לערוך את הקיים, ולכן `===`! הוא תשובה מדויקת לשאלה "השתנה משהו". הכללית: השהיה של 300 מילישניות, **וחובה** לרוקן אותה ב- `pagehide`, אחרת יש חלון שבו השינוי על המסך ואינו על הדיסק.

שלוש בדיקות של שלושים שניות, לפני שאתה מגיש

מה עושים	מה חייב לקרות
מוסיפים פריט, סוגרים את הלשונית, פותחים מחדש	הוא שם
מוחקים תו אחד מהערך במפתח ב-DevTools, ומרעננים	העמוד עולה
מנקים הכול, ומרעננים	עדיין ריק, ועם משפט